API Security Risk Analysis Report

For

JSONPlaceholder API
https://jsonplaceholder.typicode.com

Prepared by:
Luthando Gongwana

Aprile 2, 2026

Contents

API Security Risk Analysis Report 1
Executive Summary 3
Scope & Methodology 4
Target API 4
Assessment Type 4
Tools Used 4
Endpoints Tested 4
Testing Methodology 4
Missing Authentication (High Severity) 5
Description: 5
Evidence: 5
Business Impact: 5
Remediation: 5
Authorization Bypass / IDOR (High Severity) 6
Description: 6
Evidence: 6
Missing Rate Limiting (High Severity) 8
Description: 8
Evidence: 8
Business Impact: 8
Remediation: 8
Excessive Data Exposure (Medium-High Severity) 9
Description: 9
Evidence: 9
Business Impact: 9
Remediation: 9
Weak Input Validation (Medium Severity) 10
Description: 10
Evidence: 10
Business Impact: 10
Remediation: 10
Recommendations 11

Executive Summary

This report presents findings from a security risk assessment of the JSONPlaceholder API. Five security risks were identified.

High Severity Risks (3):

- Missing Authentication

- Authorization Bypass (IDOR)

- Missing Rate Limiting

Medium Severity Risks (2):

- Excessive Data Exposure

- Weak Input Validation

KEY BUSINESS IMPACT:

- Any attacker can access all user data without login
- Customer PII exposed (names, emails, phone, addresses)
- GPS coordinates reveal user locations
- No protection against brute force or DoS attacks
- GDPR/CCPA compliance violations risk

All tests were conducted as read-only security analysis with no exploitation attempts

Scope & Methodology

Target API

Name: JSONPlaceholder

URL: https://jsonplaceholder.typicode.com

Type: Public REST API for testing

Assessment Type

Read-only security analysis

No exploitation or bypass attempts

No DoS or flooding tests

Tools Used

- Postman: API requests and response inspection
- Browser DevTools: Header analysis
- Google Docs: Documentation

Endpoints Tested

Endpoint	Method	Purpose
/users	GET	Data exposure analysis
/users{id}	GET	Authorization testing
/posts	GET	Rate limiting testing
/posts	POST	Input validation testing

Testing Methodology

1. Review API documentation
2. Test endpoints using Postman
3. Inspect headers and responses
4. Identify security gaps
5. Classify risk severity
6. Document findings with evidence

Missing Authentication (High Severity)

Description:

The API accepts requests without any form of authentication. No API keys, tokens, OAuth, or credentials are required to access any endpoint.

Evidence:

- Inspected request headers in Postman - no Authorization, X-API-Key, or Bearer tokens present
- Successfully accessed all endpoints without providing any credentials
- All requests returned 200 OK without authentication

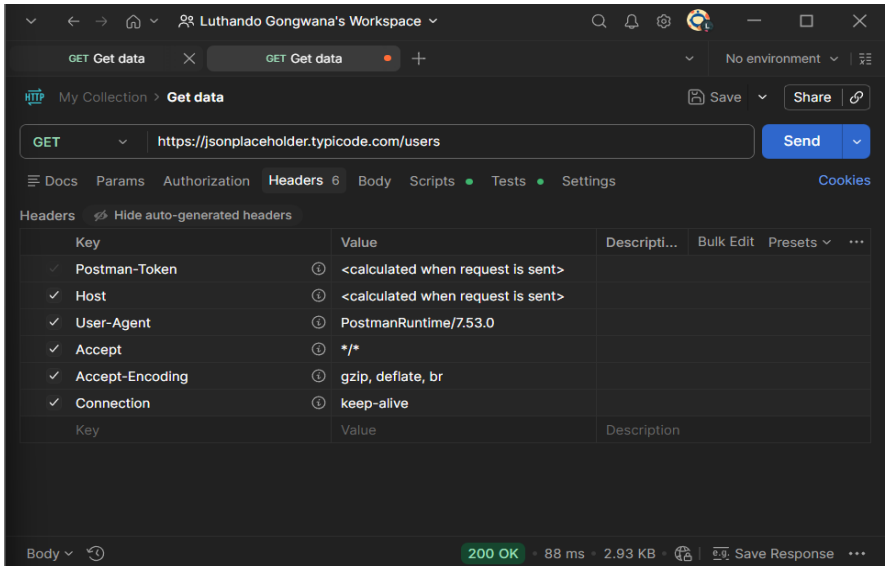

FIGURE 1 N0-AUTH

Business Impact:

- Anyone on the internet can access all API data
- No audit trail for data access
- Cannot block malicious actors
- Complete lack of security controls
- If this was production, all customer data would be publicly accessible

Remediation:

- Implement API key authentication for all endpoints
- Add OAuth 2.0 for user-specific operations
- Enforce authentication at API gateway level
- Never expose endpoints without authentication in production

Authorization Bypass / IDOR (High Severity)

Description:

The API allows access to any user's data by simply changing the ID parameter in the URL. I did not perform ownership verification.

Evidence:

- Accessed /users/1 - returned Leanne Graham's data.
- Accessed /users/3 - returned Clementine Bauch's data.
- Accessed /users/7 - returned Kurtis Weissnat's data.
- All without authentication or authorization.

Screenshot References:

FIGURE 2 USER3

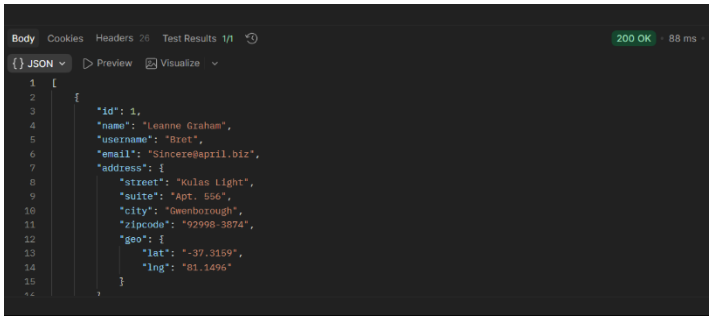

FIGURE 3USER1

FIGURE 4USER7

Business Impact:

- Attackers can enumerate all users by guessing sequential IDs.
- Complete data breach of all customer records.
- In a banking app, this would expose all account details and transactions.
- Violates data privacy regulations (GDPR Article 32 - Security of Processing).

Remediation:

- Implement access control checks on every request.
- Verify the requesting user owns the resource before returning data.
- Use UUIDs (e.g., 550e8400-e29b-41d4-a716-446655440000) instead of sequential IDs.
- Never trust client-side input for authorization decisions.

Missing Rate Limiting (High Severity)

Description:

The API accepts unlimited requests without any throttling or blocking mechanisms which then attackers can use brute force by sending many requests at a time.

Evidence:

- Sent 50+ rapid GET requests to /posts
- All requests returned 200 OK
- No 429 (Too Many Requests) responses observed
- No delays or blocking detected

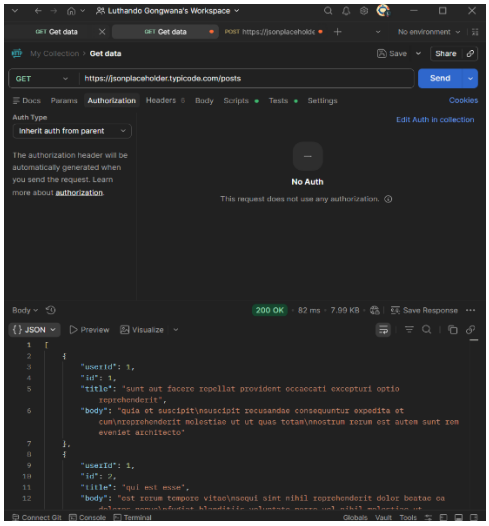

FIGURE 5 NO RATE LIMITING

Business Impact:

- Vulnerable to Denial of Service (DoS) attacks - attackers can crash the API.
- Can be abused for brute force attacks to guess passwords or enumerate data.
- Potential for high cloud infrastructure costs from excessive usage.
- No protection against automated scraping.

Remediation:

- Implement rate limiting (e.g., 100 requests per minute per IP).
- Return HTTP 429 status code when limits are exceeded.
- Add exponential backoff for repeated violations.
- Use API gateway for unified rate limiting across all endpoints.

Excessive Data Exposure (Medium-High Severity)

Description:

The API returns more data than necessary for typical use cases, including sensitive Personally Identifiable Information and GPS coordinates and company details.

Evidence:

GET /users/1 returned 12+ fields (including):

- Full name
- Email address
- Phone number
- Complete street address
- GPS coordinates (latitude and longitude)
- Company details

FIGURE 6 ALL USER DATA

Business Impact:

- Privacy violation
- Unnecessary data leakage if breached
- Increased attack surface
- GPS coordinates reveal precise user locations - serious privacy concern
- Potential regulatory fines

Remediation:

- Implement field-level filtering - only return data needed for each use case
- Use GraphQL or any software for precise data requests
- Create separate endpoints for different data needs:
- /users/public - for public-facing data
- /users/private - for authenticated users only
- Never return GPS coordinates unless if it is necessary to do so

Weak Input Validation (Medium Severity)

Description:

The API accepts malformed or invalid data without proper validation. No schema validation is enforced on write operations.

Evidence:

- Sent POST request to /posts with invalid data:
  - userId: "not-a-number" (should be integer)
  - title: "" (empty string)
  - body: null
- API returned 201 Created instead of rejecting with 400 Bad Request
- Response body echoed back the invalid data, confirming acceptance

Business Impact:

- Database may become corrupted with invalid data types
- Attackers could inject malicious payloads (SQL injection, XSS)
- Application logic may break when reading malformed data
- Poor data quality affects business reporting and decisions

Remediation:

Implement strict JSON schema validation using libraries like Joi or JSON Schema Validate data types, formats, and required fields before processing Return HTTP 400 Bad Request with descriptive error messages:

```
{
  "error": "Validation failed",
    "details": {
    "userId": "Must be an integer",
    "title": "Cannot be empty"
  }
}
```

So never accept data that doesn't match the expected schema.

Recommendations

Priority 1 - Immediate (Within 1 Week)

1. Implement Authentication

- Add API keys for all endpoints
- Implement OAuth 2.0 for user operations
- Enforce at gateway level

2. Add Authorization Checks

- Verify ownership before returning data
- Replace sequential IDs with UUIDs
- Never trust client-side input for access control

3. Deploy Rate Limiting

- Limit to 100 requests/minute per IP
- Return 429 status when exceeded
- Add exponential backoff

Priority 2 - Short-term (Within 1 Month)

4. Reduce Data Exposure

- Implement field-level filtering
- Remove GPS coordinates from responses
- Create separate endpoints for different data needs

5. Add Input Validation

- Implement JSON schema validation
- Validate all fields before processing
- Return descriptive 400 error messages